Auditor：面向机会型资源的会计处理

作者：Michael Boehler, Anton J. Gamel, Stefan Kroboth, Benjamin Rottler, Dirk Sammel and Markus Schumacher

摘要

高能物理（HEP）中日益增长的计算需求以及对高性能/高吞吐量计算能效的日益关注，推动着人们寻找更有效利用现有资源的方法。由于避免资源闲置是实现高效率的关键，适当的措施是将低利用率站点的空闲资源共享给完全occupied 的站点。COBalD/TARDIS 软件能够以自动、透明、动态的方式，以机会型模式（非）整合此类资源。然而，资源共享也需要会计处理。

在这项工作中，我们介绍 AUDITOR（面向机会型资源的会计数据处理工具箱），这是一个灵活且可扩展的会计系统，能够覆盖广泛的使用场景和基础设施。AUDITOR 通过所谓的收集器来收集会计数据，这些收集器旨在监控批处理系统、COBalD/TARDIS、云调度器或其他信息来源。数据存储在数据库中，并提供给所谓的插件，插件根据会计记录执行操作。例如，操作可以是创建所使用资源的账单、计算 CO_2足迹、调整服务的参数，或将会计信息转发给其他会计系统。根据使用场景，可以从不断增长的收集器和插件生态系统中选择合适的收集器和插件。为了便于社区开发收集器和插件，提供了与 AUDITOR 交互的库。

1. 引言

在当前关于提高可持续性和追求最大能效的讨论中，将利用率不足的计算集群中的机会型资源整合到具有持续计算任务的计算集群中的方法正变得越来越有吸引力。COBalD 和 TARDIS 软件工具的组合已被证明对于动态整合和归还机会型资源非常有用。它能够在提供机会型资源的批处理系统中生成所谓的“drone”。该批处理系统作为底层批处理系统（UBS）。然后，drone 的资源在另一个集群（即覆盖批处理系统，OBS）上可用。基于这些资源的使用情况，drone 的数量会动态调整。如果有更多资源可用，并且 drone 的利用率超过某个阈值，则会启动更多 drone。

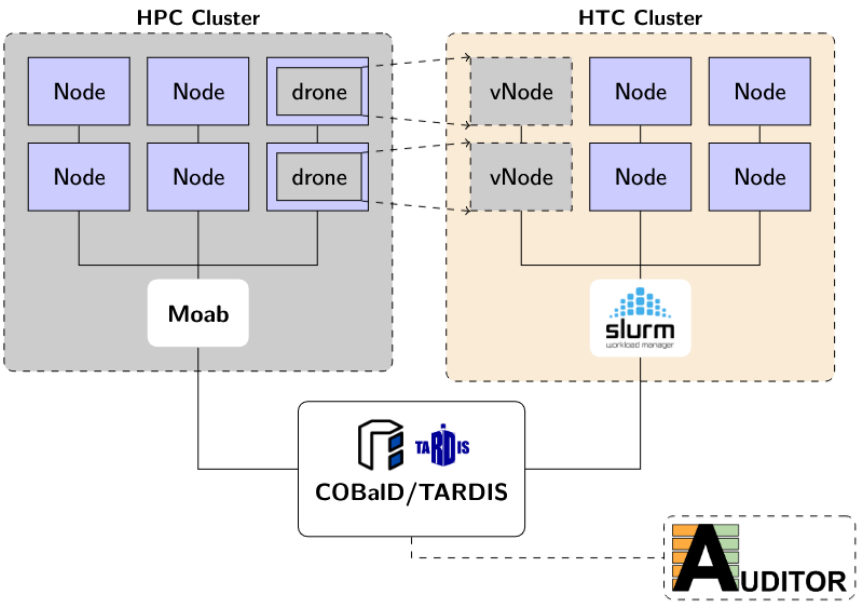

图 1.通过 COBalD/TARDIS 连接的两个集群示例。HPC 集群作为底层批处理系统（UBS），为覆盖批处理系统（OBS）（即 HTC 集群）提供资源（作为虚拟机，vNode）。COBalD/TARDIS 监控 drone 利用率并动态增加/减少 drone 数量。AUDITOR 负责对所使用资源进行适当的会计处理。

否则，不会启动新的 drone，并将资源归还给 UBS。图 1 显示了通过 COBalD/TARDIS 连接的两个集群示例。这里，UBS 是一个 HPC 集群，为 OBS（即 HTC 集群）提供额外资源。HPC 集群使用 MOA 调度器运行，HTC 集群使用 Slurm 运行。由于 Slurm 是一个传统的批处理系统，只能处理具有 IP 地址的资源，因此 drone 包含虚拟机（vNode），这些虚拟机在运行后会连接到 Slurm。然后，它们作为额外的 vNode 出现在 OBS 中。

如果多个工作组从 HPC 集群向 HTC 集群提供资源，并同时使用两个集群的资源，那么如果一个组使用的资源多于其提供的资源，就会出现不公平共享。为了防止这种情况，可以使用独立的会计实例来测量每个工作组提供的资源，并相应调整 OBS 中的组优先级。另一个用例是 HTC 集群贡献给全球 LHC 计算网格（WLCG）。这些集群必须满足特定要求：必须部署专用的计算单元，并且必须在 WLCG 工作流管理系统中注册特定队列。将可用资源动态整合到现有 WLCG 计算集群中的方法避免了部署额外中间件组件。然而，上述设置并未充分考虑机会型资源的会计处理。需要一个适当的会计系统，以便根据提供的资源进行单独的会计处理。本文提出的方法——开源软件 AUDITOR（面向机会型资源的会计数据处理工具箱）——将适应非常不同的基础设施和许多潜在用例。因此，它被设计并实现为一个高度模块化、灵活且易于扩展的架构。

2.AUDITOR 会计生态系统

AUDITOR 生态系统由三种不同类型的组件构成：核心组件（将所谓的记录存储在 PostgreSQL 数据库中）、收集器（从数据源收集数据并将其存储为记录）以及插件（基于存储的信息执行特定操作，见图 2）。收集器和插件都可以通过 REST API 向 AUDITOR 存储和检索数据。为了便于扩展 AUDITOR 生态系统，提供了两种编程语言（Rust 和 Python）的客户端库。Python 客户端库是构建在 Rust 客户端库之上的轻量级 Python 层，使用 PyO3 库实现。

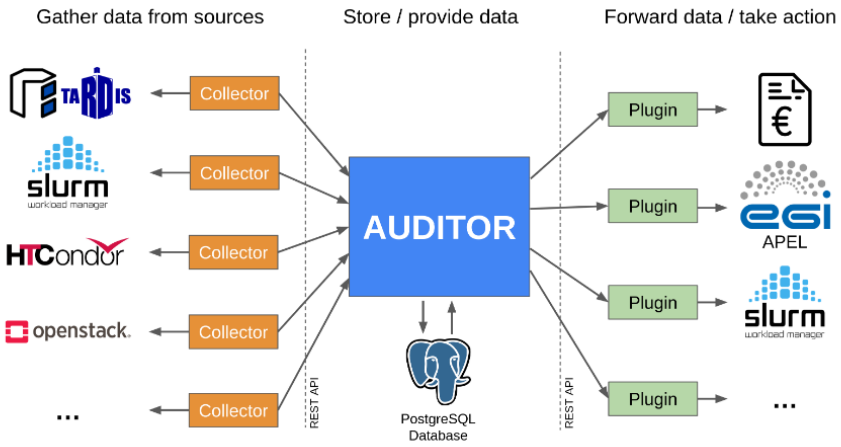

图 2：AUDITOR 生态系统概览。AUDITOR 接受来自收集器的记录并将其存储在 PostgreSQL 数据库中。它允许插件访问这些记录，插件可以根据存储的信息执行操作。

2.1 AUDITOR 核心组件

AUDITOR 的核心组件使用 Rust 编程语言编写。Rust 是一种开源、多范式的系统编程语言，旨在确保安全、并发和实用。安全性特别指避免导致内存访问错误或缓冲区溢出，从而可能产生安全漏洞（尤其是在并发进程中）的编程错误。AUDITOR 将其所有数据存储在 PostgreSQL 数据库中。通过 sqlx 库建立与 PostgreSQL 数据库的连接，该库支持编译时 SQL 查询验证和 SQL 表迁移。数据通过使用 ACTIX WEB 库实现的 REST API 进行访问。由于核心组件本身是无状态的，因此它对于数据丢失具有相当的鲁棒性，并且非常适合在负载均衡器后的多个组件组成的高可用性设置中使用。为了确保快速简便的安装，AUDITOR 是通过 CI 流水线以 RPM 软件包管理器和 Docker 容器两种格式被提供出来的。

记录代表 AUDITOR 中的一个可会计单元。它包含如下信息：record_id（唯一标识符）；meta 字段，可用于提供记录的上下文。这是一个 HashMap，其中一个键（字符串）映射到一个字符串数组。图 3 显示了一个示例记录：其中 group、site 和 user id 存储在 meta 字段中。任何数量的可在会计中考虑的属性都可以存储在 components 字段中。它至少包含一个名称和一个数量（应考虑哪种资源以及多少资源用于会计）。由于典型的计算作业会使用 CPU 和内存资源，因此这两种组件都可以考虑用于会计。可以为单个组件分配多个评分。例如，图 3 中的记录同时支持 HEPSPEC06 和新的 hepscore23 指标。start_time 和 stop_time 字段存储资源使用的时间段。从 AUDITOR 接收的数据还包含一个 runtime 字段。这是 stop_time 和 start_time 之间的差值。数据库中的所有日期和时间均为 UTC 时间。


```
{
    "record_id": "hpc-4126142",
    "meta": {
      "group_id": [ "atlpr"],
      "site_id": [ "hpc" ],
      "user_id": [ "atlpr001"  ]
    },
    "components": [
      {
        "name": "Cores",
        "amount": 8,
        "scores": [
          {
            "name": "HEPSPEC06",
            "value": 10.0
          },
          {
            "name": "HEPScore23",
            "value": 10.0
          }
        ]
      },
      {
        "name": "Memory",
        "amount": 16000,
        "scores": []
      }
    ],
    "start_time": "2023-02-24T00:27:58Z",
    "stop_time": "2023-02-24T03:41:35Z",
    "runtime": 11617
},
```


图 3：示例记录的 JSON 表示。meta 字段可以包含记录的上下文。components 指明应考虑哪种资源以及多少资源用于会计。

2.2 收集器

收集器的任务是从各种数据源收集相关的会计数据，并将其以记录的形式传输到 AUDITOR 的 REST 端点。本文介绍了两个 Slurm 收集器和一个 TARDIS 收集器。一个 HTCondor 收集器正在开发中。

为了从 Slurm 批处理系统收集会计数据，可以使用 Slurm Epilog 收集器或 Slurm 收集器。Slurm Epilog 收集器基于 Slurm 的 Epilog 功能。在每个批处理作业结束时，可以在头节点或客户端节点上自动执行任何脚本。对于 Slurm Epilog 收集器，会对每个 Slur，作业执行 Slurm 命令‘scontrol show job <jobid>’，解析相关信息，转换为记录并发送给 AUDITOR。该收集器被编译成一个可移植、静态链接的二进制文件，没有系统依赖关系，并且可广泛配置，使其非常适合不同的用例和环境。Slurm Epilog 收集器是一种简单但高效的实现，用于收集 Slurm 作业的基本会计信息。由于 Epilog 收集器作为作业本身的一部分在 Epilog 中运行，因此如果解析或传输数据到 AUDITOR 出现延迟，作业运行时间乃至 CPU 效率将受到影响。此外，在 Epilog 执行期间，并非所有 Slurm 作业的会计信息都可用。Slurm 收集器则是专门为那些无法接受 Slurm Epilog 收集器这些限制的用例而实现的。

Slurm 收集器作为一个独立工具运行，使用命令行工具‘sacct’定期查询 Slurm 会计数据库中的新作业。为每个作业创建一个新记录并放入队列中，然后该队列以可配置的定期间隔将记录传输到 AUDITOR。如果传输到 AUDITOR 失败，记录将被放回队列，并在下一个间隔重试传输过程。为避免数据丢失，队列的内容也以 SQLite3 数据库的形式存储在磁盘上。该收集器也被编译成一个可移植、静态链接的二进制文件，没有系统依赖关系。Slurm 收集器也可以进行广泛配置。唯一的要求是必须安装 Slurm 客户端软件。

如第 1 节所述，COBalD/TARDIS 管理着一组作为机会型资源的 drone。每个 drone 可以处于有限数量的状态之一（例如，启动中、运行中、已停止等）。TARDIS 仅跟踪这些状态之间的转换。每个状态转换事件也会转发到 TARDIS 的插件接口，TARDIS 收集器会连接到该接口。由于只传输状态变更，完整的记录只能在 drone 终止后创建。为了避免收集器必须跟踪每个 drone 从创建到终止的全过程，TARDIS 收集器使用 AUDITOR 的/add 端点为 drone 创建一个不完整的记录，然后在 drone 终止后使用/update 端点将 stop_time 添加到记录中。这降低了收集器的复杂性，但增加了收集器与 AUDITOR 之间的交互次数，因为每个记录必须先创建后更新。该收集器与 TARDIS 软件一同交付。

2.3 插件

插件可以根据存储在 AUDITOR 数据库中的记录信息执行各种各样的任务。第一个可用的插件是优先级插件，当两个集群通过 COBalD/TARDIS 连接时，它能够根据底层批处理系统中提供的资源来操作覆盖批处理系统调度器中的组优先级。AUDITOR-APEL 插件能够将 AUDITOR 中收集的会计信息转发到 EGI APEL 会计平台。另一个可能的用例是一个利用率报告工具，它分析每个用户作业请求的资源与消耗的资源，并提供每周概览，其中包含潜在的节省量和相应的 CO_2足迹，以提高用户的意识。最后两个插件在撰写本文时尚未就绪。接下来，将详细介绍优先级插件。

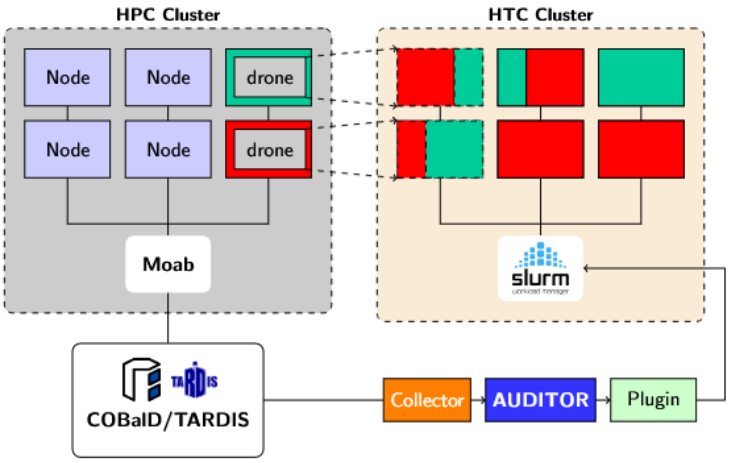

图 4.红组和绿组为 HTC 集群提供了高性能计算集群的资源。该优先级插件能够确保在 HTC 集群中，作业的优先级会根据每个组所提供的资源量进行相应调整。

3.使用例示例－根据提供的资源调整优先级

弗莱堡大学已实现了对 HPC 资源的机会型使用，具体如下：高能物理（HEP）的四个工作组可以访问本地的 NEMO HPC 集群。每个工作组的服务帐户可以根据需要在 HPC 集群上请求资源（作为 drone），并将其作为共享资源提供给所有四个 HEP 工作组的用户在覆盖批处理系统（OBS）中使用。这些共享资源即为所提供的资源。

由于各个工作组既可以直接在 HPC 集群上计算，也可以通过 OBS 机会型地进行计算，因此必须确保各个工作组 OBS 中的优先级根据其提供的资源进行调整。他们向 OBS 贡献的资源越多，其优先级就越高。这一目标是通过一个合适的收集器、一个专用的 AUDITOR 实例以及专门开发的优先级插件来实现的。

TARDIS 收集器从 drone 中收集每个工作组所提供的资源信息，并将记录存储在 AUDITOR 实例中。优先级插件从 AUDITOR 获取必要的数据，并计算每个工作组[A,...,D]在过去 14 天内提供的资源 ci（以 vCore 小时计）：

$$c_i = \int_{t_{now}-14d}^{t_{now}} N_i(t)dt \qquad (1)$$

其中，Ni(t)Ni(t)是在时间实例（t）下，与组 i∈{A,B,C,D}相关联的所有记录所提供的核心数量。优先级（pi）定义为：单个组提供的资源量（ci）与所有 HEP 工作组提供的 vCore 小时总量$\sum_j c_j$之比，再乘以设定的最小优先级值 p_{min} 与最大优先级值 p_{max} 之间的差值，即：

$$p_i = \frac{c_i}{\sum_j c_j} \cdot (p_{max} - p_{min}) + p_{min} \qquad (2)$$

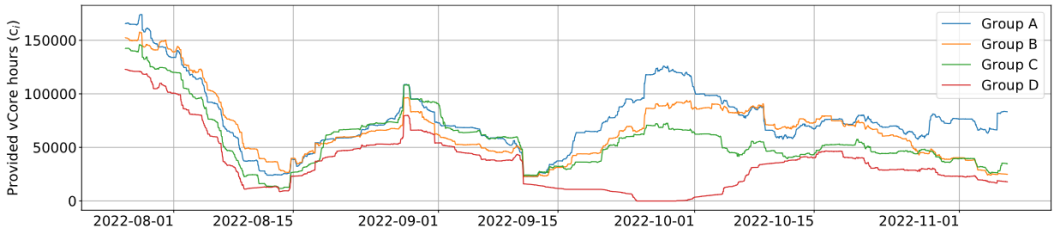

图 5. 根据公式 1，2022 年 8 月至 11 月期间在 NEMO 集群上记录的每个组(i)过去 14 天的虚拟核心小时积分值(c_i（t））。

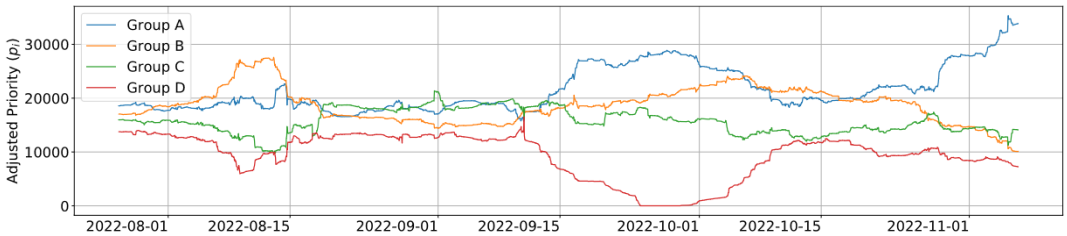

图 6. 根据公式 2，基于 2022 年 8 月至 11 月期间记录的 NEMO 作业计算出的每个组(i)的优先级(p_i（t））。

其中 i∈A,B,C,D。组优先级 p_i 每天在 OBS（在此用例中为 Slurm 调度器）上进行调整。最小和最大优先级分别设置为 p_{min}=1 和 p_{max}=65335。图 4 显示了四个工作组中两个组的作业，分别用绿色和红色表示。两个组都在 HPC 集群中生成了一个 drone。红色组在 HTC 集群中使用两个完整的工作节点，并与绿色组共享另外 3 个工作节点的份额。尽管 HPC 集群采用单用户每节点的策略，但 OBS 中的虚拟节点可以用作共享资源。这提高了 CPU 效率，因为单个用户的任务通常无法填满 HPC 集群中的一个完整节点。图 5 和图 6 显示了以虚拟核心小时数衡量的、在 HPC 集群上分配的资源，以及根据公式 2 计算出的新组优先级。可以看出，在 2022 年 10 月初，D 组未在 HPC 集群上提供任何资源，因此其组优先级被最小化。由于 Slurm 调度器还会考虑等待时间和其他标准，D 组并未被完全禁止使用 OBS，但大幅降低的组优先级导致 D 组成员提交的作业等待时间显著增加。

4. 结论

AUDITOR 是一个高度模块化且灵活的会计生态系统。其 REST 接口以及 Rust 和 Python 客户端库允许广大用户社区为生态系统的扩展做出贡献，并快速轻松地实现自己的收集器和插件。

现有的收集器和插件可以任意组合，以实现各种用例。如果某个特定用例无法用现有组件实现，则可以轻松扩展以涵盖分布式计算系统中作业会计上下文中的任何其他用例。

5. 致谢

这项工作得到了联邦教育与研究部（BMBF）在项目 05H21VFRC2 “Entwicklung, Integration und Optimierung von digitalen Infrastrukturen für ErUM”框架内的支持，该研究属于合作研究中心“Föderierte Digitale Infrastrukturen für die Erforschung von Universum und Materie (FIDIUM)”。

弗莱堡的 NEMO HPC 集群得到了巴登-符腾堡州科学、研究和艺术部通过 bwHPC 项目的资助，以及德国研究基金会（DFG）通过资助号 INST 39/963-1 FUGG 的支持。

参考文献　略，不做翻译

主要观点：

这份论文介绍了**一个开源的软件系统——Auditor**，这个软件专为高能物理领域而设计，主要用于对“机会型资源”进行会计处理。

论文首先阐明了现实背景，由此带出了研究的意义：高能物理学等计算密集型领域对计算和能效的需求不断增长。目前的普遍处理方式是使用“机会型资源”进行“借调”，然而现有的、可以完成这种借调的处理软件却无法对这些动作进行会计核算与处理，因此 Auditor 软件被设计出来，用于解决这个问题。

其次，作者介绍了 Auditor 软件的具体构成。该软件由三个部分组成：Core component（核心），Collector（收集器）和 Plugins（插件），三者有各自的功能，形成一个整体；随后，作者以弗莱堡大学的 NEMO HPC 集群为例，说明了 AUDITOR 的工作流程；最后，作者们称赞了 Auditor 软件高度模块化和灵活的优点，并相信它能运用于多个领域。